

Introduction to business analytics

Dr.Yasser ElGeddawy

Course Overview

- This course introduces business analytics, focusing on data-driven decision-making.
- Learn fundamental concepts of data analysis, visualization, and statistical methods to extract insights for business strategy.
- The course includes hands-on exercises using **Excel**, **SQL**, and **Python/R**.

Course Objectives

- Understand the role and importance of business analytics.
- Collect, clean, and analyze data to make business decisions
- Apply statistical methods to business scenarios
- Use data visualization techniques to communicate insights
- Work with Excel, SQL, and basic Python/R for data analysis
- Interpret business trends using predictive analytics

Ch1- Index

- What is Business Analytics?
- Types of Analytics: Descriptive, Predictive, and Prescriptive
- Role of Data in Decision-Making
- Case Studies in Business Analytics

What are Business Analytics?

- Business Analytics (BA) is the practice of using data, statistical analysis, and machine learning techniques to make informed business decisions. It helps organizations improve efficiency, optimize operations, and gain a competitive advantage.

Scope of Business Analytics

- Business Analytics (BA) refers to the process of collecting, analyzing, and interpreting data to improve business decision-making.
- It involves a combination of data management, statistical analysis, and predictive modelling to identify patterns and trends.
- BA is used in various domains, including marketing, finance, supply chain, human resources, and operations management.

Importance of Data-Driven Decision-Making

- Data-driven decisions are based on evidence and analysis rather than intuition.
- Businesses use analytics to:
 - Identify trends and patterns in customer behavior.
 - Optimize marketing campaigns for better ROI.
 - Reduce operational costs and improve efficiency.
 - Predict future trends and mitigate risks.
- • **Example:** Retail companies use sales data to forecast demand and adjust inventory levels accordingly.

Difference Between Data Analytics, Business Intelligence, and Business Analytics

Concept	Definition	Key Focus	Example
Data Analytics	The broad process of analyzing raw data to extract insights.	Includes data collection, cleaning, and statistical analysis.	A data analyst examines customer purchase trends in an e-commerce store.
Business Intelligence (BI)	The use of tools and technologies to transform raw data into meaningful reports and dashboards.	Reporting, data visualization, and descriptive analytics.	A sales dashboard that shows revenue trends across different regions.
Business Analytics (BA)	The application of statistical methods and predictive models to solve business problems.	Decision-making, forecasting, and optimization.	A bank uses BA to predict which customers are likely to default on loans.

Types of Analytics: Descriptive, Predictive, and Prescriptive

- Business analytics can be classified into three main types: **Descriptive**, **Predictive**, and **Prescriptive Analytics**.
- Each type serves a unique purpose in decision-making and helps businesses leverage data effectively.

Descriptive Analytics: Understanding Past Performance

- **Definition:**

- Descriptive analytics focuses on analyzing historical data to understand what
- has happened in the past.
- It involves summarizing raw data into meaningful insights using reports,
- dashboards, and visualizations.

- **Techniques Used:**

- Data aggregation
- Data visualization (charts, graphs, dashboards)
- Statistical measures (mean, median, variance, etc.)

- **Examples in Business:**

- **Sales Reports:** A retail company analyzes past sales trends to understand customer demand for different products.
- **Website Traffic Analysis:** Google Analytics provides insights into website visitor behavior and engagement.
- **Customer Segmentation:** A company groups customers based on demographics and past purchasing behavior.

Predictive Analytics: Forecasting Future Trends

- **Definition:**

- Predictive analytics uses statistical models and machine learning algorithms to predict future outcomes based on historical data.
- It helps businesses anticipate trends, identify risks, and make data-driven forecasts.

- **Techniques Used:**

- Regression analysis (linear, logistic)
- Time series forecasting
- Machine learning models (decision trees, neural networks)

- **Examples in Business:**

- **Customer Churn Prediction:** A telecom company predicts which customers are likely to cancel their subscriptions.
- **Fraud Detection:** Banks use predictive models to identify unusual transaction patterns and prevent fraud.
- **Demand Forecasting:** E-commerce companies predict product demand to optimize inventory management.

Prescriptive Analytics: Recommending Actions Based on Insights

- **Definition:**

- Prescriptive analytics goes beyond predicting outcomes by providing actionable recommendations.
- It suggests the best course of action using optimization algorithms and AI-based decision-making models.

- **Techniques Used:**

- Optimization models
- Decision trees
- Artificial Intelligence (AI) & Machine Learning

- **Examples in Business:**

- **Pricing Optimization:** Airlines and hotels use prescriptive models to dynamically adjust prices based on demand.
- **Supply Chain Optimization:** Companies determine the most efficient way to manage inventory and logistics.
- **Healthcare Treatment Plans:** AI-driven analytics recommend personalized treatment plans for patients.

Comparison of Descriptive, Predictive, and Prescriptive Analytics

Type of Analytics	Purpose	Key Question Answered	Example
Descriptive Analytics	Analyzes past data	What happened?	A company analyzes last quarter's sales data to identify trends.
Predictive Analytics	Forecasts future trends	What is likely to happen?	A bank predicts the likelihood of a loan applicant defaulting.
Prescriptive Analytics	Suggests best actions	What should we do?	Uber uses AI to recommend the best routes for drivers based on traffic patterns.

Role of Data in Decision-Making

- Data plays a critical role in modern business decision-making by enabling organizations to optimize operations, enhance customer experiences, and increase profitability.
- Companies that leverage data effectively gain a competitive advantage over those that rely solely on intuition.

How Businesses Use Data to Improve Efficiency and Profitability

- Businesses use data analytics to enhance decision-making in various ways:
- **Operational Efficiency:**
 - Manufacturing companies use predictive maintenance to reduce equipment downtime.
 - Logistics firms optimize delivery routes using real-time GPS data.
- **Marketing Optimization:**
 - E-commerce platforms analyze customer behavior to improve product recommendations.
 - Digital marketers track campaign performance to allocate ad budgets effectively.
- **Customer Experience Enhancement:**
 - Streaming services (e.g., Netflix) personalize content recommendations.
 - Retailers use past purchase data to improve loyalty programs.

Data-Driven vs. Intuition-Based Decision-Making

Approach	Definition	Pros
Data-Driven Decision-Making (DDDM)	Using data, analytics, and statistical methods to guide business decisions.	- More accurate and objective

Data-Driven

- Reduces risks and biases
- Enables predictive forecasting
- Requires quality data
- May need technical expertise
- Can be time-consuming

Intuition-Based

- Relying on personal experience, gut feeling, or instincts.
- Quick decision-making
- Works in uncertain situations
- No need for large datasets
- Subjective and prone to bias
- Higher risk of poor decisions
- Lacks measurable evidence

Example:

- A retailer deciding which products to stock can either:
 - ✓ **Use data** from past sales trends and customer preferences (**data-driven**)
 - X Rely on instinct based on experience (**intuition-based**)

The Importance of Data Quality, Accuracy, and Governance

For business analytics to be effective, data must be **accurate, consistent, and well-governed**. Poor data quality can lead to incorrect insights and costly mistakes.

- **Data Quality Factors:**

- Accuracy: Ensuring data is free of errors and reflects real-world values.
- Completeness: Avoiding missing or incomplete data records.
- Consistency: Data should be uniform across all databases and reports.
- Timeliness: Data must be up to date for effective decision-making.

- **Data Governance Practices:**

- Implementing data privacy policies (e.g., GDPR, CCPA).
- Establishing data ownership and access control to protect sensitive data.
- Conducting regular audits to maintain data integrity.

- **Example:** A financial institution must ensure its customer data is accurate to avoid errors in loan approvals and credit scoring.

Ethical Considerations in Business Analytics

- As businesses increasingly rely on data, ethical concerns must be addressed:
- **Data Privacy:**
 - Protecting customer data from unauthorized access.
 - Complying with regulations like GDPR (Europe) and CCPA (California).
- **Bias in Analytics:**
 - AI models should be designed to prevent discrimination (e.g., hiring algorithms must be fair).
- **Transparency:**
 - Companies should disclose how customer data is collected and used.
- **Responsible AI and Automation:**
 - Ensuring automated decision-making does not harm consumers (e.g., fair pricing algorithms).

Example: A credit scoring model should not disadvantage applicants based on gender, ethnicity, or location.

Key Takeaways

- Data-driven decision-making enhances efficiency and profitability.
- Poor data quality can lead to misleading insights and bad business decisions.
- Ethical data governance is critical for protecting privacy and preventing bias.

Case Studies in Business Analytics

Retail Industry: How Amazon Uses Predictive Analytics for Personalized Recommendations

- **Problem:**

- Amazon needed a way to increase customer engagement and sales by
- recommending relevant products to users.

- **Solution:**

- Amazon implemented a **collaborative filtering algorithm** that analyzes customer
- browsing and purchasing behavior to provide **personalized recommendations**.

- **How It Works:**

- Uses machine learning to identify patterns in customer behavior.
- Recommends products based on **past purchases, search history, and similar users' activities**.
- Dynamically updates recommendations in real time.

Retail Industry: How Amazon Uses Predictive Analytics for Personalized Recommendations

- **Impact:**

- Increased sales through cross-selling and upselling.
- Enhanced customer experience by providing relevant product suggestions.
- Boosted **35% of Amazon's revenue** from personalized recommendations.

Example:

- If a customer buys a laptop, Amazon might recommend accessories like a **laptop bag, external hard drive, or wireless mouse.**

Finance: Fraud Detection Using Machine Learning in Banks

- **Problem:**

- Banks face increasing challenges in detecting fraudulent transactions and
- minimizing financial losses.

- **Solution:**

- Banks use **machine learning and predictive analytics** to **detect unusual patterns in transactions** that may indicate fraud.

- **How It Works:**

- Uses historical transaction data to train fraud detection models.
- Identifies anomalies, such as sudden large **transactions** or **unusual spending patterns**.
- Implements **real-time fraud alerts** and transaction blocking.

Finance: Fraud Detection Using Machine Learning in Banks

- **Impact:**

- Reduced fraud losses by detecting suspicious transactions early.
- Improved customer trust by preventing unauthorized activities.
- Increased efficiency in fraud detection, reducing reliance on manual reviews.

Example:

- If a customer's credit card is suddenly used in a different country, the bank's
- fraud detection system may flag the transaction and send a verification alert.

Common Python Libraries for Data Analysis

1. NumPy:

- What it Does: Provides support for handling large, multi-dimensional arrays and performing mathematical computations.
- Example Use Case: Calculating the mean of a dataset stored in an array.

2. pandas:

- What it Does: A robust tool for manipulating, cleaning, and analyzing structured data (e.g., tables).
- Example Use Case: Removing duplicates and filling missing values in a dataset.

3. Matplotlib:

- What it Does: Creates static, animated, and interactive plots for data visualization.
- Example Use Case: Drawing a line graph to show sales trends over time.

Common Python Libraries for Data Analysis

4. **seaborn:**

- What it Does: Builds on Matplotlib, offering enhanced and aesthetically pleasing statistical visualizations.
- Example Use Case: Creating heatmaps to visualize correlations between variables.

5. **scikit-learn:**

- What it Does: Provides tools for building and evaluating machine learning models.
- Example Use Case: Using regression models to predict future sales or trends.

Python Basics

Variables and Data Types

- What Are Variables?
- Variables are containers for storing data values.
- You can declare a variable by assigning a value to it using the = operator.

Example:

```
name = "Alice" # String
age = 25        # Integer
height = 5.7    # Float
is_student = True # Boolean
```

Common Data Types

- **Integer:** Whole numbers (e.g., 10, -5).
- **Float:** Numbers with decimals (e.g., 3.14, -0.99).
- **String:** Text enclosed in quotes (e.g., "Hello, World!").
- **Boolean:** Logical values (True or False).
- **List:** A collection of items (e.g., [1, 2, 3] or ["apple", "banana", "cherry"]).

Common Data Types

Type Conversion:

- Converting one data type to another using built-in functions like `int()`, `float()`, `str()`.

Example:

```
age = "25" # String  
age = int(age) # Converted to Integer
```

Basic Operations

- **Arithmetic:** +, -, *, /, // (floor division), % (modulus), ** (exponentiation).
- **String Concatenation:** + to join strings.

```
greeting = "Hello" + ", " + "Alice!"  
print(greeting)  # Output: Hello, Alice!
```


Control Structures

- Conditional Statements (if, elif, else):
 - Used to execute code based on conditions.

Example:

```
age = 18
if age < 18:
    print("You are a minor.")
elif age == 18:
    print("You are exactly 18!")
else:
    print("You are an adult.")
```

Loops

- 1. for Loops:
 - Iterate over a sequence (list, range, string, etc.).
- Example:

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Loops

- 2. while Loops:
 - Repeat if a condition is True.
- Example

```
count = 0
while count < 5:
    print(count)
    count += 1  # Increment count
```

Functions

- Defining Functions:
 - Functions are defined using the def keyword.

Example:

```
def greet(name):  
    return f"Hello, {name}!"  
print(greet("Alice")) # Output: Hello, Alice!
```

Functions

- Function Arguments:
 - Functions can accept inputs (arguments).
- Example:

```
def add_numbers(a, b):  
    return a + b  
  
print(add_numbers(3, 4))  # Output: 7
```

- Return Values:
 - Functions can return results using the return keyword.

Importance of Modularity and Reusability

- Functions allow breaking complex problems into smaller, manageable parts.
- Code can be reused by calling the function multiple times.

Example:

```
def calculate_area(length, width):  
    return length * width  
print(calculate_area(5, 3)) # Output: 15
```

End of Lecture 1

Introduction to Python Libraries for Data Analysis

Dr.Yasser ElGeddawy

Ch2- Index

1. Overview of Key Libraries
2. Explain the importance of NumPy for Working with Arrays and Numerical Data.
3. Demonstrate pandas: Data Structures (Series and DataFrame) and Basic Operations
4. Importing and Exporting Data

What is NumPy?

- NumPy (Numerical Python) is a powerful library for numerical computations in Python.
- It provides support for multi-dimensional arrays and a vast collection of mathematical functions to operate on these arrays efficiently.

Key Features

1. Efficient Storage:

- Optimized for handling large datasets and reducing memory usage.

2. Mathematical Operations:

- Functions such as mean, median, standard deviation, and matrix operations.

3. Element-Wise Operations:

- Perform operations on arrays element by element without writing explicit loops.

Statistical Functions in NumPy

- **NumPy** is a core Python library for numerical computing, built for handling large arrays and matrices efficiently.
- NumPy provides a robust set of functions to analyze data. Here they are categorized by their purpose:

NumPy: Statistical Functions

1. Order Statistics

- These are the "Big Three" of descriptive statistics, used to find the center and spread of your data.

Function	Definition
<code>np.amin()</code>	Finds the minimum value in an array or along a specific axis.
<code>np.amax()</code>	Finds the maximum value in an array or along a specific axis.
<code>np.percentile()</code>	Computes the q^{th} percentile of the data (e.g., the value below which 50% of the data falls).
<code>np.quantile()</code>	Similar to percentile, but uses a scale of 0 to 1 instead of 0 to 100.
<code>np.ptp()</code>	"Peak-to-Peak" — calculates the range (Maximum - Minimum).

NumPy: Statistical Functions

2. Averages and Variances

Function	Definition
<code>np.median()</code>	Finds the middle value of the array (ignoring outliers better than the mean).
<code>np.mean()</code>	Calculates the arithmetic average of the elements.
<code>np.average()</code>	Calculates the average, but allows for weights (giving some numbers more importance than others).
<code>np.std()</code>	Calculates the Standard Deviation , measuring how spread out the numbers are from the mean.
<code>np.var()</code>	Calculates the Variance , which is the square of the standard deviation.

NumPy: Statistical Functions

3- Correlation and Histograms

Used to understand the relationship between different datasets or the distribution of values.

Function	Definition
<code>np.corrcoef()</code>	Returns the Pearson product-moment correlation coefficients (how two variables relate).
<code>np.cov()</code>	Estimates the covariance matrix of the data.
<code>np.histogram()</code>	Computes the occurrence frequency of data points within specific "bins" or ranges.

NumPy: Statistical Functions

4- Merging and sorting

Operation	Function	Goal	Result
Sorting	<code>np.sort()</code>	Rank data	Same size array, different order.
Merging	<code>np.concatenate()</code>	Combine data	Larger array, same original order.

NumPy: Statistical Functions

Using the axis Parameter

- Most of these functions allow you to specify an axis.
- `axis=0`: Performs the calculation down the columns.
- `axis=1`: Performs the calculation across the rows.

Example:

`np.mean(matrix, axis=0)` will give you the average of each column individually.

Basic Operations

Simple Example

```
import numpy as np

group1 = np.array( [75,30,60,31,55] )
group2 = np.array( [61,73,89,92,100,78] )
values = np.concatenate( [group1,group2] )
print("Mean: ", np.mean(values))
print("Median: ", np.median(values))
print("Standard Deviation: ", np.std(values))
print("Variance: ", np.var(values))
print("Maximum: ", np.max(values))
print("Minimum: ", np.min(values))
print("Sorted Grades:\n", np.sort(values))
```

Mean: 67.63636363636364

Median: 73.0

Standard Deviation: 21.96804591497191

Variance: 482.595041322314

Maximum: 100

Minimum: 30

Sorted Grades:

[30 31 55 60 61 73 75 78 89 92 100]

Creating Arrays

- You can create arrays using the `np.array()` method.

Here are some examples:

```
import numpy as np

# Create a 1D array
array1 = np.array([1, 2, 3, 4])
print(array1)

# Create a 2D array
array2 = np.array([[1, 2], [3, 4]])
print(array2)
```

Common Operations

1- Element-Wise Addition: Add a constant to every element in the array:

```
array1 + 2  
# Output: array([3, 4, 5, 6])
```

2. Calculating Mean: Use `np.mean()` to calculate the mean of an array:

```
np.mean(array1)  
# Output: 2.5
```

Indexing and Slicing

- Access specific elements or slices of an array.

```
# Access first element of a 1D array  
print(array1[0]) # Output: 1  
  
# Access an element in a 2D array (row 0, column 1)  
print(array2[0, 1]) # Output: 2
```

- **Slicing:** Extract subarrays:

```
print(array1[1:3]) # Output: [2, 3]  
print(array2[:, 1]) # Output: [2, 4] (all rows, column 1)
```

Example: 1D array

- To understand these functions, it is helpful to imagine a real-world scenario. Let's use a dataset of **exam scores** for a class of students.
- Some students' scores are more important than others (a "weighted" average), which allows us to see how each function behaves differently.
- Let's look up the code via VS code.

Visualizing the Difference

It is often confusing to know when to use Mean vs. Median.

- **Mean:** Good for "normal" data where everything is similar.
- **Median:** Best when you have outliers (like a student scoring 0 or 100 in a group of 50s).
- **Standard Deviation:** If this number is low, the students all scored similarly. If it is high, the class has a huge gap between the top and bottom students.

Example

:.2f part is called a Format Specifier. It is a shortcut used inside an "f-string" (formatted string) to control how a number looks when you print it.

Why do we use it?

Without it, Python might print a number with 10 or 15 decimal places, which is messy and hard to read.

Code	Output	Why?
<code>print(weighted_avg)</code>	88.55555555555556	Default Python behavior.
<code>print(f"{weighted_avg:.2f}")</code>	88.56	Rounds to 2 decimal places.
<code>print(f"{weighted_avg:.1f}")</code>	88.6	Rounds to 1 decimal place.
<code>print(f"{weighted_avg:.0f}")</code>	89	Rounds to the nearest whole number.

Example: 2D array

- **Working with 2D Data (Rows vs. Columns)**
- If you have a matrix of scores (Rows = Students, Columns = Subjects), you can calculate these statistics for each subject specifically:

```
# 3 students, 2 subjects (Math, Science)
class_scores = np.array([
    [80, 90],
    [70, 85],
    [95, 100]
])

# Average for each subject (axis 0 = vertical/columns)
subject_means = np.mean(class_scores, axis=0)
print(f"Mean for Math: {subject_means[0]}, Mean for Science: {subject_means[1]}")
```

Exercise

1. Create a 1D NumPy array with numbers from 1 to 10.
2. Perform the following:
 - Multiply all elements by 3.
 - Calculate the mean and standard deviation of the array.
3. Create a 2D NumPy array (3x3 matrix) and extract:
 - The first row.
 - The last column.

End of Lecture 2

Introduction to Python Libraries for Data Analysis

Dr.Yasser ElGeddawy

Pandas: Data Structures (Series and DataFrame) and Basic Operations

What is pandas?

- pandas is a powerful Python library designed for data manipulation and analysis.
- It offers flexible and easy-to-use data structures, enabling you to clean, analyze, and visualize data efficiently.

Key Data Structures

1. Series:

- A 1D labelled array, like a column in a spreadsheet.
- Each data point has an associated label (index).

2. DataFrame:

- A 2D labelled data structure, like an Excel table or SQL table.
- Composed of rows and columns.

Key Features

1. Handling Missing Data:

- Detect and fill missing values easily.

2. Data Transformation:

- Grouping, merging, pivoting, and reshaping datasets.

3. File I/O:

- Import and export data in formats like CSV, Excel, and JSON.

Series

1. Creating a Series:

```
import pandas as pd

# Create a Series with Labeled indices
data = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
print(data)
```

Output:

```
a    1
b    2
c    3
d    4
dtype: int64
```

Series

2. Accessing Elements:

- By label:

```
print(data['b']) # Output: 2
```

- By position:

```
print(data[1]) # Output: 2
```

DataFrame

- 1- Creating Data frames:

```
import pandas as pd
# Create a simple DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

DataFrames

2. Accessing Data:

- Access a column:

```
print(df["Age"])
```

- Output

```
0    25
1    30
2    35
Name: Age, dtype: int64
```

- Access a Row
- Access a row using .loc:

```
print(df.loc[1]) # Access the second row (index 1)
```

- Output

```
Name    Alice
Age      25
City    New York
Name: 0, dtype: object
```

- The `df.loc[]` property in the Python **pandas** library is used for **label-based indexing** to access a group of rows and columns from a DataFrame

DataFrames

```
[6] df.loc[df["A"] > 50]
```



	A	B	C
5	60	90	120
6	70	100	130
7	80	110	140
8	90	120	150
9	100	130	160



```
[18] df.loc["row2":"row4"]
```



	A	B
row2	30	80
row3	40	90
row4	50	100



Difference between .Loc and .iLoc

- The primary difference between .loc and .iloc in Pandas is that:
- .loc is **label-based** (using index/column names),
- while .iloc is **integer-position-based** (using 0-based numerical indices).

Difference between .Loc and .iLoc

- **.loc (Label-based):**
 - Uses row and column labels (df.loc['row_name'col_name']).
 - **Inclusive:** Slicing df.loc['a':'c'] includes both 'a' and 'c'.
 - Supports boolean arrays for filtering.
- **.iloc (Integer-based):** Uses integer positions (df.iloc[0, 2]).
 - **Exclusive:** Slicing df.iloc[0:2] includes index 0 and 1, but not 2.
 - Does not support boolean arrays, only integers and slices.

When to Use Which?

Use .loc when you know the specific names of your rows or columns. Use .iloc when you need to select data based on its position in the DataFrame (e.g. "first 5 rows" "last column")

Difference between .Loc and .iLoc

- loc — Supports passing a condition as a filter directly, selecting rows or columns where the condition is true.

Example — Select rows where values in column “A” are greater than 50.

- iloc — This method does not support passing a condition like `df["A"] > 50`. It only works with integer positions, so you'd need to use integer indexing explicitly, rather than filtering by conditions.

Difference between .Loc and .iLoc

Feature	.loc	.iloc
Type of Indexing	Label-based indexing (uses actual names)	Integer-based indexing (uses indices)
Slicing	Uses labels of rows and columns	Uses integer positions of rows and columns
Boolean Indexing Support	Supports direct Boolean indexing	Does not support direct Boolean indexing
Inclusive Indexing	Both start and end labels are included	Start index is included, end index is excluded

Basic DataFrame Operations

- **1. Descriptive Statistics:**
- Use `.describe()` for quick summary statistics:

```
print(df.describe())
```

Output:

	Age
count	3.0
mean	30.0
std	5.0
min	25.0
25%	27.5
50%	30.0
75%	32.5
max	35.0

Basic DataFrame Operations

2-Add a New Column:

- Output:

```
df["Salary"] = [50000, 60000, 70000]  
print(df)
```

	Name	Age	City	Salary
0	Alice	25	New York	50000
1	Bob	30	Los Angeles	60000
2	Charlie	35	Chicago	70000

Importing and Exporting Data

Reading Data from Files

1. CSV Files

- CSV (Comma-Separated Values) files are commonly used for storing tabular data.
- Use `pandas.read_csv()` to load data from a CSV file into a DataFrame.

```
import pandas as pd

# Import a CSV file
data = pd.read_csv("data.csv")
print(data)
```

Reading Data from Files

2. Excel Files For Excel files

- use `pandas.read_excel()` to import data.

```
# Import an Excel file  
data = pd.read_excel("data.xlsx")  
print(data)
```

End of Lecture 3

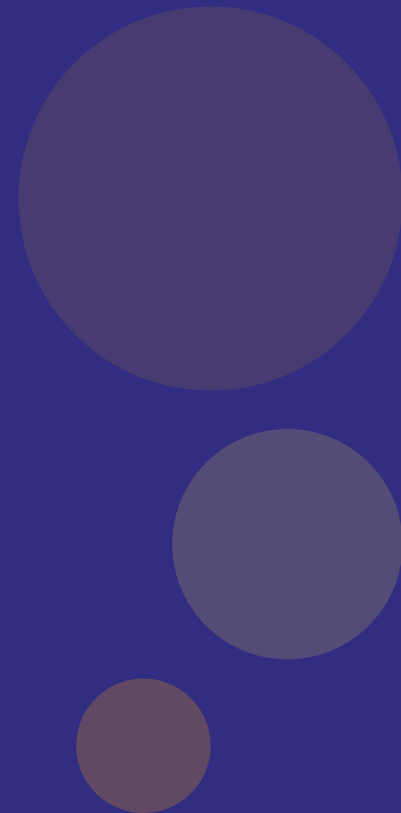
Python · Pandas · NumPy · CSV

Data Cleaning & Preprocessing

in Python

A structured module covering data quality, cleaning pipelines, and preprocessing techniques using pandas and numpy.

Dr. Yasser ElGeddawy



Learning Objectives

What you will achieve in this Chapter

01

Understand the Importance of Data Cleaning

Why clean data matters for analysis and ML



02

Handle Missing Data Effectively

Detect, impute, fill, and drop null values



03

Data Type Conversion & Handle Duplicates

Cast types correctly; identify and remove duplicates



04

Rename, Drop, and Filter Data

Reshape datasets: rename columns, drop rows, apply filters



05

Apply String Operations and Formatting

Clean and normalize text data with pandas str methods



The Python Data Cleaning Toolkit

Three essential libraries — one powerful workflow

pandas

```
import pandas as pd
```

Data manipulation & analysis

- DataFrame and Series structures
- Read/write CSV, Excel, JSON
- GroupBy, merge, reshape
- Built-in missing-value handling

numpy

```
import numpy as np
```

Numerical computing engine

- Fast array operations
- np.nan for missing values
- Statistical functions
- Backbone of pandas internals

CSV Files

```
pd.read_csv('data.csv')
```

Universal data exchange format

- Comma-Separated Values — plain text
- Most common data source format
- Easy to inspect and share
- Supports encoding, separators, headers

Objective 1 — Understand the Importance of Data Cleaning

1

Garbage In, Garbage Out

Dirty data produces unreliable results, regardless of algorithm quality.

2

ML Models Break on NaN

Most sklearn estimators raise errors if NaN values are present in features.

3

Wrong Types = Silent Bugs

Numeric data stored as strings won't aggregate; dates won't sort correctly.

4

Duplicates Skew Stats

Duplicate rows inflate counts, averages, and can bias trained models.

5

Inconsistent Strings Break Joins

'Cairo', 'cairo', 'CAIRO' become three groups instead of one.

```
df.info() → see dtypes, non-null counts, memory
```

80%

of data science time
spent cleaning data

3×

better model accuracy
with clean data

\$15M

avg. annual loss from
poor data quality

Loading CSV Files — Your First Step

loading_csv.py

```
import pandas as pd
import numpy as np

# -- Basic load -----
df = pd.read_csv("customers.csv")

# -- Load with options -----
df = pd.read_csv(
    "customers.csv",
    sep          = ",",          # delimiter
    header       = 0,           # row 0 = header
    na_values    = ["NA", "?", "-"],
    parse_dates = ["signup_date"],
    encoding     = "utf-8",
    dtype       = {"id": str}   # force str
)

# -- First-look inspection -----
print(df.shape)           # (rows, cols)
print(df.head(3))         # first 3 rows
print(df.dtypes)          # column types
print(df.info())          # full summary
print(df.describe())      # numeric stats
```

sep / delimiter

Use `sep='\t'` for tab-separated, `sep=';'` for European CSVs.

parse_dates

Pass column names to auto-convert date strings to datetime.

na_values

Specify custom missing-value tokens beyond the defaults.

dtype

Force column types at load to avoid silent coercion errors. (e.g., strings becoming NaN when pandas guesses wrong)



Objective 2 — Handle Missing Data Effectively

① DETECT

```
# Count nulls per column
# * 100 Converts the fraction into a
percentage.
df.isnull().sum()
df.isnull().mean() * 100

# Rows with any null
df[df.isnull().any(axis=1)]

# np.nan checks
import numpy as np
np.isnan(df['score'])

# Heatmap (seaborn)
import seaborn as sns
sns.heatmap(df.isnull(),
            cbar=False)
```

② DROP

```
# Drop rows with ANY null
df.dropna(inplace=True)

# Drop if col ≥ 50% missing
thresh = int(len(df) * 0.5)
df.dropna(
    axis=1, thresh=thresh)

# Drop specific columns
#errors= ignore, input validation so
it won't throw errors if the columns
don't exist.
df.drop(
    ['old_col', 'junk'],
    axis=1, inplace=True,
    errors='ignore')

# Reset index after drop
df.reset_index(
    drop=True, inplace=True)
```

③ FILL

```
# Fill with scalar
df.fillna(0, inplace=True)

# Fill numeric with mean
df['age'].fillna(
    df['age'].mean())

# Fill with mode (categorical)
df['city'].fillna(
    df['city'].mode()[0])

# Forward / backward fill
#Forward fill (ffill): Fills NaN with
the last non-null value above it.
#Backward fill (bfill): Fills NaN with
the next non-null value below it.
#inplace=True modifies the DataFrame
directly without creating a copy.

df.ffill(inplace=True)
df.bfill(inplace=True)
```



Key rule: Always investigate WHY data is missing before choosing a strategy — missing data can be informative!

Objective 3 — Data Type Conversion and Handle Duplicates

Data Type Conversion

Pandas dtype	Python type	Use case
object	str	Text / mixed
int64	int	Whole numbers
float64	float	Decimals
bool	bool	True / False
datetime64	datetime	Dates & times
category	Categorical	Repeated strings

```
# to_numeric - coerce bad values to NaN
df['price'] = pd.to_numeric(df['price'],
                           errors='coerce')

# to_datetime
df['date'] = pd.to_datetime(df['date'],
                           format='%Y-%m-%d')

# astype shortcuts
df['qty'] = df['qty'].astype(int)
df['status'] = df['status'].astype('category')
print(df.dtypes)
```

Handling Duplicates

```
# -- Detect -----
# Count total duplicates
df.duplicated().sum()

# View all duplicate rows
df[df.duplicated(keep=False)]

# -- Remove -----
# Drop duplicates (keep first)
df.drop_duplicates(inplace=True)

# Keep last occurrence
df.drop_duplicates(keep='last',
                  inplace=True)

# Subset of columns
df.drop_duplicates(
    subset=['email', 'phone'],
    inplace=True)

# -- After dropping -----
df.reset_index(drop=True,
               inplace=True)
print(f"Rows after dedup: {len(df)}")
```

Objective 4 — Rename, Drop, and Filter Data

RENAME

Improves readability; required before merging mismatched datasets.

```
# Rename specific columns
df.rename(columns={
    'cust_nm': 'customer_name',
    'dt':      'date',
    'amt':     'amount'
}, inplace=True)

# Standardise ALL column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_')
)
```

DROP

Remove irrelevant features and rows that fail quality checks.

```
# Drop single column
df.drop('phone', axis=1,
        inplace=True)

# Drop multiple columns
df.drop(['col1', 'col2'],
        axis=1, inplace=True)

# Drop rows by label index
df.drop([0, 5, 12],
        axis=0, inplace=True)

# Drop rows by condition
df.drop(
    df[df['age'] < 0].index,
    inplace=True)
```

FILTER

Extract targeted subsets for analysis using conditional expressions.

```
# Boolean mask
df[df['salary'] > 5000]

# Multiple conditions (&, |)
df[(df['age'] >= 18) &
    (df['status'] == 'active')]

# isin() filter
cities = ['Cairo', 'Giza']
df[df['city'].isin(cities)]

# query() – readable syntax
df.query(
    "salary > 5000 and "
    "dept == 'Sales'"
)

# Between
df[df['score'].between(70, 100)]
```

Objective 5 — Apply String Operations and Formatting

`.str.strip()`

Remove leading/trailing whitespace

`.str.lower()` / `.upper()`

Normalise case uniformly across rows

`.str.replace(pat, rep)`

Replace substring or regex pattern

`.str.contains(pat)`

Boolean mask — rows matching pattern

`.str.split(pat, expand=True)`

Split into separate DataFrame columns

`.str.extract(r'(group)')`

Pull regex capture groups as columns

str.replace

- `r'^[a-z\s]'` will only keep lowercase English letters and spaces.
- If you want to keep uppercase letters too, you should use `A-Z`.
- If you want to keep digits as well, you should add `0-9`.
- `.str.replace()` with `regex=True` is correct for regex-based replacement. (regex= regular expression)

Practical — Clean a Name Column

```
df = pd.read_csv("customers.csv")

# Step 1 -- strip whitespace
df['name'] = df['name'].str.strip()

# Step 2 -- lowercase
df['name'] = df['name'].str.lower()

# Step 3 -- remove special chars
df['name'] = df['name'].str.replace(
    r'^[a-z\s]', '', regex=True)

# Step 4 -- collapse extra spaces
df['name'] = df['name'].str.replace(
    r'\s+', ' ', regex=True).str.strip()

# Step 5 -- extract first / last name
# n=1 → splits only on the first space, so last names with spaces
# remain intact.
# .expand=True → returns a DataFrame instead of a Series

df[['first', 'last']] = (
    df['name']
    .str.split(' ', n=1, expand=True))
```



Key rule: **pat** → The substring or regular expression **pattern** you want to find. **rep** → The string (or callable) you want to **replace** it with.

Full Cleaning Pipeline — Putting It All Together

pipeline.py

```
import pandas as pd
import numpy as np

# 1. Load -----
df = pd.read_csv("raw_data.csv",
                 na_values=["NA", "?", "-"])

# 2. Inspect -----
print(df.info())
print(df.isnull().sum())

# 3. Rename columns -----
df.rename(columns={'cust_nm': 'name',
                  'dt': 'date',
                  'amt': 'amount'},
          inplace=True)
df.columns = df.columns.str.lower()

# 4. Handle missing -----
#.fillna('') → prevents errors if there are NaN values.
#.dropna() → Removes rows (or columns) that contain missing values (NaN or None).
df['amount'].fillna(df['amount'].mean(),
                  inplace=True)
df.dropna(subset=['name', 'date'],
          inplace=True)
```

1

Load

pd.read_csv with na_values

2

Inspect

df.info() & isnull().sum()

3

Rename

rename() + standardise columns

4

Missing

fillna(), dropna() by strategy

5

Types

to_datetime, to_numeric, astype

6

Duplicates

drop_duplicates + reset_index

7

Strings

str.strip, str.lower, str.title

8

Export

to_csv(index=False)

Full Cleaning Pipeline — Putting It All Together

pipeline.py

```
# 5. Convert types -----
df['date'] = pd.to_datetime(df['date'])
df['amount'] = pd.to_numeric(df['amount'],
                             errors='coerce')
df['status'] = df['status'].astype('category')

# 6. Duplicates -----
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)

# 7. String cleaning -----
df['name'] = (df['name'].str.strip()
              .str.lower()
              .str.title())

# 8. Export -----
df.to_csv("clean_data.csv", index=False)
print("Done:", df.shape)
```

1

Load

pd.read_csv with na_values

2

Inspect

df.info() & isnull().sum()

3

Rename

rename() + standardise columns

4

Missing

fillna(), dropna() by strategy

5

Types

to_datetime, to_numeric, astype

6

Duplicates

drop_duplicates + reset_index

7

Strings

str.strip, str.lower, str.title

8

Export

to_csv(index=False)

Quick Reference Cheat Sheet

pandas · numpy · CSV — the essential commands

1 Load

```
pd.read_csv('data.csv')
```

2 Inspect

```
df.info() /  
df.describe()
```

3 Missing

```
df.isnull().sum() →  
fillna/dropna
```

4 Types

```
pd.to_numeric /  
pd.to_datetime / astype
```

5 Duplicates

```
df.drop_duplicates()
```

6 Rename

```
df.rename(columns={...})
```

7 Drop

```
df.drop(['col'], axis=1)
```

8 Filter

```
df[df['col'] > val] /  
df.query()
```

9 Strings

```
.str.strip().str.lower()  
.str.replace()
```

10 Export

```
df.to_csv('clean.csv',  
index=False)
```

End Of Lecture 4

CHAPTER 4

Exploratory Data Analysis (EDA)

Dr. Yasser ElGeddawy

Topics: Descriptive Statistics • Sorting & Ranking • Grouping & Aggregation • Pivot Tables

Learning Objectives

1

Understand the Purpose of EDA

Explain how EDA helps uncover patterns, trends, and relationships in data.

2

Apply Descriptive Statistics

Calculate key measures like mean, median, and variance using pandas.

3

Sort and Rank Data

Identify outliers, trends, or specific rankings in datasets.

4

Grouping and Aggregation

Use `groupby()` to categorize data and apply aggregation functions.

5

Create Pivot Tables

Construct pivot tables to summarize data efficiently.

What is Exploratory Data Analysis?

EDA is the process of analysing datasets to summarize their main characteristics — often using visual and statistical methods — before applying formal modelling.

EDA Process



Uncover Patterns

Detect trends, clusters, and recurring structures in data.



Spot Anomalies

Find outliers and unexpected values that may need attention.



Check Assumptions

Validate statistical assumptions before modelling.



Find Relationships

Discover correlations between variables.

4.1 Descriptive Statistics — Key Measures



Mean

The average value of the dataset.

→ Ages [25,30,35,40,50] → Mean = 36



Median

The middle value in a sorted dataset.

→ Ages [25,30,35,40,50] → Median = 35



Mode

The value that appears most frequently.

→ Scores [80,90,80,70] → Mode = 80



Std Dev

Spread of data around the mean.

→ Low std = consistent data



Variance

Square of standard deviation.

→ High variance = wide spread in values

4.1.2 Summary with describe() in pandas

```
df.describe()
```

```
import pandas as pd
data = {
    'Age': [25,30,35,40,50],
    'Salary': [50000,60000,
              70000,80000,60000]
}
df = pd.DataFrame(data)
print(df.describe())
```

Stat	Age	Salary
count	5	5
mean	36.0	64,000
std	9.62	11,401
min	25	50,000
25%	30	60,000
50%	35	60,000
75%	40	70,000
max	50	80,000

Key Outputs:

count: Number of non-missing values

mean: Average of all values

std: Spread around the mean

25/50/75%: Quartile distribution markers

min/max: Extreme values in the dataset

4.1.3 Identifying Trends & Patterns



Central Tendency

Use mean, median, and mode to find the 'center' of the data.

Example:

Mean salary of \$60,000 → suggests typical earnings.



Consistency

Standard deviation & variance show how much values deviate.

Example:

High std dev in salary → unequal pay distribution.



Outlier Detection

Data points far from the mean or median may be anomalies.

Example:

Ages mostly 20–40, but one entry is 100 → likely error.

4.2 Sorting and Ranking Data

Sorting with `sort_values()`

by=Column(s) to sort by

ascending=True (asc) / False (desc)

inplace=Modify original or create copy

```
# Sort by Age (ascending)
df.sort_values(by='Age')

# Sort by Salary (descending)
df.sort_values(by='Salary',
               ascending=False)
```

Ranking with `rank()`

average: Avg rank for tied values (default)

min: Smallest rank to tied values

max: Largest rank to tied values

```
# Rank salaries (highest = 1)
df['Rank'] = df['Salary'].rank(
    ascending=False)

# Result: 80000→1, 70000→2
# 60000→3, 50000→4
```

Why Sorting & Ranking Matter in EDA

Sorting: Highlights extremes, prioritizes data for decisions (e.g., top sales regions). **Ranking**: Enables comparisons, builds leaderboards, measures relative performance.

4.3 Grouping and Aggregating Data

The groupby() Function

groupby() splits data into groups based on one or more columns. Combined with aggregation functions, it provides focused insights on subsets of data.

mean()

Average of values

Avg salary by dept

sum()

Total sum

Total revenue by region

max()

Maximum value

Highest sale per month

min()

Minimum value

Lowest price per category

count()

Number of values

Orders per customer

```
# Group by Department, get avg salary
grouped = df.groupby('Department')
result = grouped['Salary'].mean()
print(result)
```

```
# Multiple aggregations
df.groupby('Region').agg({
    'Sales': 'sum',
    'Profit': 'mean'
})
```

Practical Use Cases:

Sales by Region → calculate total sales per region

Avg Rating by Product Category → find top categories

Revenue by Month → identify peak seasons

4.3 Grouping and Aggregating Data - Complete Example

Here's a **complete, runnable Python example** showing how to use `groupby()` in Pandas to:

1. **Group by a single column** and get the average salary.
2. **Perform multiple aggregations** on different columns.

```
import pandas as pd

# Sample data
data = {
    'Department': ['HR', 'IT', 'IT',
                  'Finance', 'Finance', 'HR'],
    'Salary': [50000, 60000, 65000, 70000,
              72000, 52000],
    'Region': ['East', 'West', 'East',
              'West', 'East', 'West'],
    'Sales': [200, 300, 250, 400, 350,
             150],
    'Profit': [20, 30, 25, 40, 35, 15]
}

# Create DataFrame
df = pd.DataFrame(data)

# 1 Group by Department and get average salary
grouped = df.groupby('Department')
avg_salary = grouped['Salary'].mean()
print("Average Salary by Department:")
print(avg_salary)
print()
```

```
# 2 Multiple aggregations: sum of Sales and mean of Profit by Region
agg_result =
df.groupby('Region').agg({
    'Sales': 'sum',
    'Profit': 'mean'
})
print("Aggregated Sales and Profit by Region:")
print(agg_result)
```

Output

Average Salary by Department:

Department

Finance 71000.0

HR 51000.0

IT 62500.0

Name: Salary, dtype: float64

Aggregated Sales and Profit by Region:

Sales Profit

Region

East 800 26.67

West 850 28.33

Key Points:

`groupby('Department')['Salary'].mean()` → Groups by **Department** and calculates the mean salary.

`.agg({...})` → Allows **different aggregation functions** for different columns.

The result of `groupby()` is a **new DataFrame or Series** depending on the operation.

4.4 Introduction to Pivot Tables

Pivot tables summarize data across two dimensions (rows + columns), quickly calculating metrics like sums, averages, or counts.

`pivot_table()` Parameters

values: The column to aggregate

index: Rows of the pivot table

columns: Columns of the pivot table

aggfunc: Aggregation function (default: mean)

```
pivot = df.pivot_table(
    values='Salary',
    index='Department',
    columns='Region',
    aggfunc='mean'
)
print(pivot)
```

Example: Avg Sales by Product & Region

Product \ Region	North	South	East
Laptop	\$1200	\$1100	\$1300
Phone	\$800	\$750	\$820
Tablet	\$500	\$480	\$530

Pivot Tables vs. Grouped Data

Pivot Table: Two-dimensional summary (rows & columns)

groupby(): One-dimensional grouped aggregation

Best use: Pivot for cross-tabulations, groupby for series

Note: You can replace 'mean' with 'sum', 'count', 'max', 'min', or even a custom function.

Chapter 4 — Key Takeaways



Descriptive Statistics

Mean, median, mode, std dev and variance provide an essential overview of data structure, trends, and anomalies.



Sorting & Ranking

`sort_values()` organizes data; `rank()` measures relative performance. Together they highlight extremes and enable comparisons.



Grouping & Aggregation

`groupby()` + aggregation functions (`sum`, `mean`, `count`) reveal patterns within subsets — critical for business insights.



Pivot Tables

`pivot_table()` gives a two-dimensional cross-tabulation, ideal for analysing trends by multiple categories simultaneously.

CHAPTER 5

Data Visualization

-Matplotlib and seaborn

(Part 1)

Dr. Yasser ELGeddawy

Topics: Matplotlib • seaborn • Line Plots • Bar Charts • Scatter Plots • Customization

Learning Objectives

1 Understand Data Visualization

Grasp the importance of visualization in EDA and data storytelling.

2 Learn Matplotlib & seaborn

Use both libraries to create a range of visualizations in Python.

3 Create Basic Plot Types

Build line plots, bar charts, and scatter plots from scratch.

4 Customize Plots

Add titles, labels, legends, styles, and gridlines for clarity.

Why Data Visualization?

Visualization is a critical tool for exploring, understanding, and presenting patterns and insights in data. It turns raw numbers into visual stories.



Explore Data

Quickly spot patterns, trends, and clusters that are hard to see in tables.



Communicate

Present insights clearly to stakeholders without technical nonsense.



Detect Anomalies

Identify outliers and unusual data points visually.



Show Relationships

Reveal correlations and dependencies between variables.



Compare Categories

Contrast groups, time periods, or products side by side.



Track Trends

Follow changes over time with time-series visualizations.

5.1.1 Matplotlib — The Foundation Library

Key Features

Foundation library

Serves as the backbone for higher-level libraries like seaborn

Highly customizable

Complete control over every aspect of a plot

Diverse plot types

Line, bar, scatter, pie, histogram, and more

Static & animated

Supports static, interactive, and animated visualizations

Basic Syntax

```
import matplotlib.pyplot as plt
```

```
# Create data  
x = [1, 2, 3, 4, 5]  
y = [2, 4, 6, 8, 10]
```

```
# Create a simple line plot  
plt.plot(x, y)
```

```
# Add labels and title  
plt.title('My First Plot')  
plt.xlabel('X-axis')  
plt.ylabel('Y-axis')
```

```
# Display the plot  
plt.show()
```

```
#To install the package run pip in terminal >>  
pip install matplotlib
```

5.1.2 seaborn — Statistical Visualization

Feature	Matplotlib	seaborn
	Pure Python, no dependency	Built on top of Matplotlib
	More verbose for complex plots	Simpler, cleaner syntax
	Manual data extraction needed	Handles pandas DataFrames directly
	Basic defaults, manual styling required	Professional defaults & color palettes
	Fine-grained control, custom plots	Statistical graphics, EDA

```
import seaborn as sns    # import seaborn
import matplotlib.pyplot as plt
sns.set_theme()          # applies clean default styles automatically
```

5.2.1 Line Plots

Purpose & Use Cases

Visualize trends, patterns, or changes over time

Often used for time-series data (e.g., monthly sales)

Can display multiple lines by calling `plt.plot()` multiple times

The `plot()` function connects data points from parameter arrays (x, y)

`xlabel()` and `ylabel()` functions add axis labels

Key Insight

Line plots are the go-to chart for any question involving change over time — sales trends, temperature variations, or stock prices.

Example Code

```
import matplotlib.pyplot as plt

# Time-series sales data
months = [1,2,3,4,5,6]
sales = [200,250,230,300,280,350]

plt.plot(months, sales)
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales ($)')
plt.show()
```

Output: Upward trending line across months



5.2.2 Bar Charts

Purpose & Use Cases

Compare quantities across different categories

Useful for categorical data analysis

`bar()` function uses rectangular bars to represent data

`bar()` takes arguments describing the layout of bars

Best when comparing discrete groups or categories

Example Code

```
import matplotlib.pyplot as plt

regions = ['North', 'South', 'East', 'West']
sales = [4500, 3200, 5100, 2800]

plt.bar(regions, sales,
        color=['#1A6B8A', '#E94560',
               '#F5A623', '#5C6BC0'])
plt.title('Sales by Region')
plt.xlabel('Region')
plt.ylabel('Sales ($)')
plt.show()
```

Simulated Output — Sales by Region



5.2.3 Scatter Plots

Purpose & Use Cases

Visualize relationships or correlations between two continuous variables

Identify trends, clusters, or outliers in data

scatter() function plots one dot per observation

Both x and y parameters take arrays of values

Useful for finding patterns before regression modelling

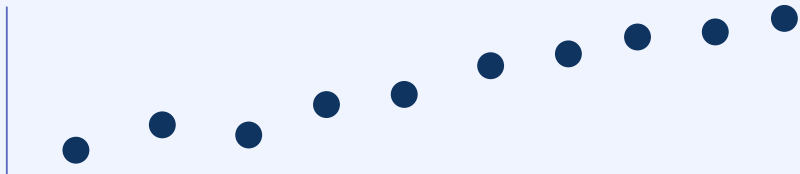
Example Code

```
import matplotlib.pyplot as plt

# Advertising vs. Sales
ad_spend = [10,20,30,40,50,
            60,70,80,90,100]
sales = [15,25,22,35,40,
         55,60,72,75,95]

plt.scatter(ad_spend, sales,
            color='steelblue', marker='o')
plt.title('Ad Spend vs. Sales')
plt.xlabel('Ad Spend ($K)')
plt.ylabel('Sales ($K)')
plt.show()
```

Simulated Output — Positive correlation between Ad Spend & Sales



Positive Correlation: Both variables increase together

Negative Correlation: One increases as other decreases

No Correlation: No visible pattern between variables

5.3 Customizing Plots

5.3.1 Labels & Titles

`plt.title('My Plot')` — set the chart title

`plt.xlabel('X-axis')` — label the x-axis

`plt.ylabel('Y-axis')` — label the y-axis

5.3.3 Styles

`plt.style.use('ggplot')` — apply a preset visual style

Options: 'seaborn', 'dark_background', 'bmh', etc.

Ensures consistent aesthetics across all charts

5.3.2 Legends

`plt.legend()` — display a legend for all labeled elements

Add `label=` to each `plot()` call to name lines

Distinguishes multiple lines or datasets on one chart

5.3.4 Gridlines

`plt.grid(True)` — add gridlines to the chart

Improves readability of data points

Helps viewers align values to axes

5.3 Customized Plot — Complete Example

```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y1 = [2,4,6,8,10]
y2 = [1,3,5,7,9]

# Apply a built-in style
plt.style.use('ggplot')

# Plot two lines
plt.plot(x, y1,
         label='Line 1',
         linestyle='--', color='blue')
plt.plot(x, y2,
         label='Line 2',
         linestyle='--', color='red')

# Customization
plt.title('Customized Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.legend()
plt.show()
```

Code Breakdown

- 1 **Imports:** `matplotlib.pyplot` imported as `plt`
- 2 **Data Prep:** `x`, `y1`, `y2` defined as lists
- 3 **Style:** `plt.style.use('ggplot')` sets aesthetics
- 4 **Plot Lines:** Two `plt.plot()` calls with `linestyle` & `color`
- 5 **Enhancements:** Title, x/y labels, gridlines, legend added
- 6 **Display:** `plt.show()` renders the final output

Output Features: Title • X/Y labels • Two colored lines with dashed/solid styles • ggplot visual style • Gridlines for readability • Legend identifying Line 1 & Line 2

Chapter 5 — Key Takeaways

Matplotlib

Foundational Python library. More verbose but provides complete control over every aspect of the visualization.

seaborn

Simplified plotting with cleaner syntax. Better defaults and integrates directly with pandas DataFrames for professional-quality plots.

Plot Types

Line plots → trends over time. Bar charts → category comparisons.
Scatter plots → relationships between continuous variables.

Customization

Titles, labels, legends, styles, and gridlines enhance clarity, readability, and make your data story more compelling.

Visualization is a critical EDA tool — it transforms raw numbers into actionable insights.

Chapter 6

Data Visualization

Part 2

Advanced Plot Types · Time Series · Best Practices

Learning Objectives

1

Understand Advanced Plot Types

Identify use cases for histograms, boxplots, heatmaps, and pairplots — and know when to use each.

2

Create Advanced Visualizations

Generate histograms, boxplots, heatmaps, and pairplots using Matplotlib and Seaborn in Python.

3

Visualize Time Series Data

Work with time-indexed data to uncover trends, seasonality, and patterns over time.

4

Apply Best Practices

Improve clarity, consistency, appropriate chart selection, annotations, and audience-centric design.

Chapter at a Glance

6.1

Advanced Visualizations

- › Histograms
- › Boxplots
- › Heatmaps
- › Pairplots

6.2

Time Series Plotting

- › Date ranges
- › Trend lines
- › Axis formatting
- › Grid & markers

6.3

Best Practices

- › Clarity
- › Consistency
- › Appropriate charts
- › Annotations
- › Audience design

Advanced Visualizations

Histograms · Boxplots · Heatmaps · Pairplots

6.1.1 Histograms

What is a Histogram?

A histogram is a graph showing frequency distributions — it displays the number of observations within each given interval (bin).

Use `plt.hist()` from Matplotlib.

Purpose:

- ✓ Display the distribution of a dataset
- ✓ Understand the frequency of values

📌 Real-world example: Exam scores of 1000 students → see if most scored near the average or if scores are skewed.

Key Parameters

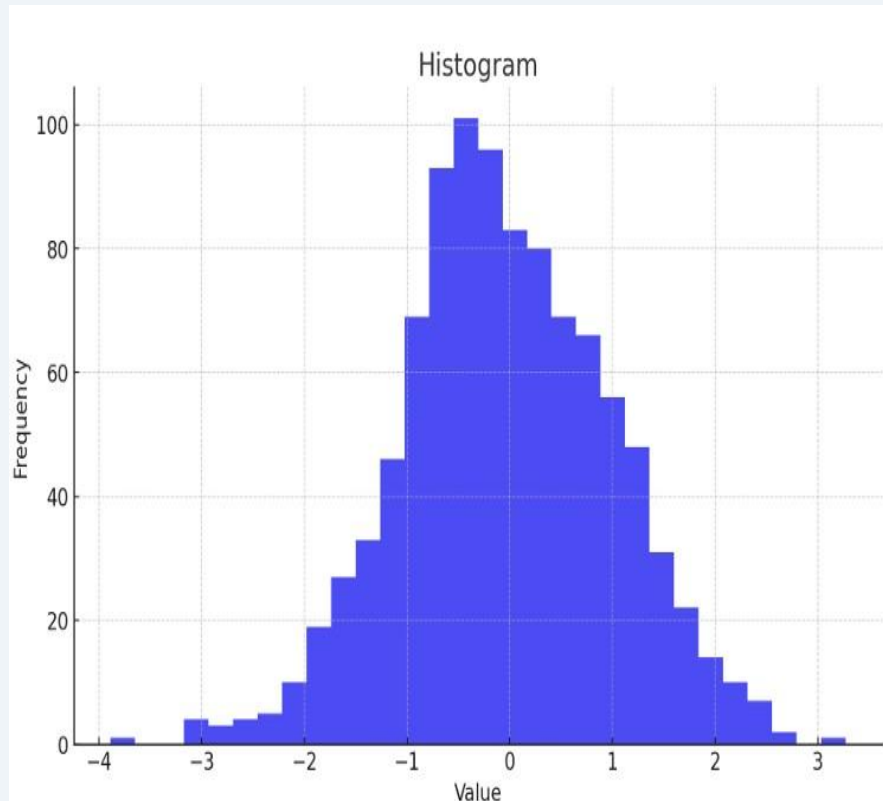
<code>data</code>	The array / dataset to visualize.
<code>bins=30</code>	Number of intervals. More bins → finer view; fewer → thicker.
<code>color='blue'</code>	Bar fill color.
<code>alpha=0.7</code>	Transparency (0 = invisible, 1 = solid). Useful when overlapping.
<code>plt.xlabel / ylabel</code>	Label the axes for clarity.
<code>plt.title()</code>	Add a descriptive title.

6.1.1 Histograms

The Code

```
import matplotlib.pyplot as plt
import numpy as np

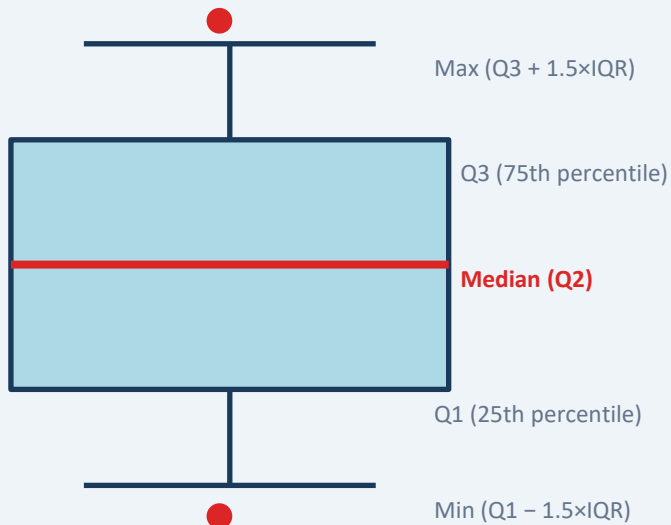
data = np.random.randn(1000) # Generate random data
plt.hist(data, bins=30, color='blue', alpha=0.7)
plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



6.1.2 Boxplots (Whisker Plots)

Outliers ●

IQR = 50%
of data



Example: Salaries in a company → boxplot immediately shows the median salary, the spread (IQR), and which employees are outliers (very high earners).

Outliers which extend to 1.5 times the IQR from the box. "If a value is more than one and a half 'box-lengths' away from the middle data, it's weird enough to be investigated." (statistical fact)

patch_artist=True: This is a "toggle" that allows you to fill the box with a color; without it, facecolor won't work.

boxprops: This dictionary allows you to style the specific box component of the plot.

Key Concepts & Parameters

Purpose

Summarize data using quartiles & detect outliers.

Interquartile range (IQR)

Box spans $Q1 \rightarrow Q3$ (middle 50% of data).

Whiskers

Extend $1.5 \times IQR$ beyond the box.

Outliers

Points outside whiskers — plotted individually.

vert=False

Horizontal boxplot instead of vertical.

patch_artist=True

Fill the box with color.

boxprops

`dict(facecolor='lightblue')` — custom appearance.

6.1.2 Boxplots (Whisker Plots)

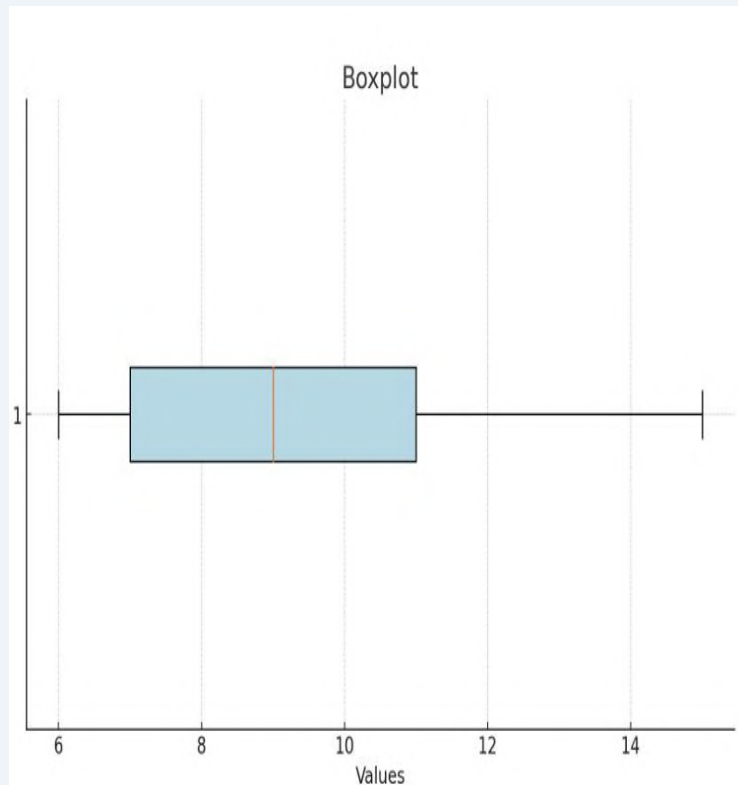
The Code

```
# Sample data
data = [7, 8, 6, 9, 12, 15, 11, 6, 10]

# Create a boxplot
plt.boxplot(
    data,
    vert=False, # Horizontal boxplot
    patch_artist=True, # Fill the box with color
    boxprops=dict(facecolor='lightblue') # Box color
)

# Add labels and title
plt.title("Boxplot Example")
plt.xlabel("Values")

# Display the plot
plt.show()
```



6.1.3 Heatmaps



What is a Heatmap?

A graphical representation of data using colors to visualize matrix values. Brighter/reddish colors represent higher values; darker colors represent lower values.

Use `seaborn.heatmap()`.

Key Parameters:

`annot=True`

Display cell values inside each box.

`cmap='coolwarm'`

Colormap: blue (low) → white → red (high).

`linewidths=0.5`

Thin lines between cells for readability.

 *Example: Correlation matrix of product features — quickly spot which variables are strongly related (dark red) or unrelated (white/pink).*

6.1.3 Heatmaps

The Code

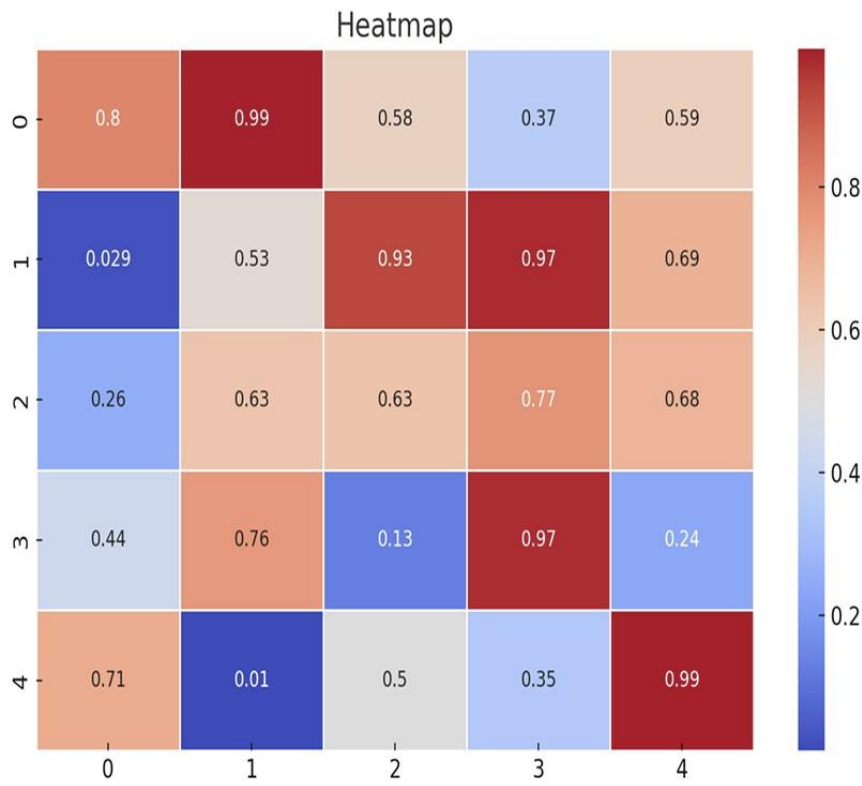
```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

# Generate a 5x5 matrix of random values
data = np.random.rand(5, 5)

# Create a heatmap
sns.heatmap(
    data,
    annot=True, # Display the data values in the cells
    cmap='coolwarm', # Set the color scheme
    linewidths=0.5 # Add spacing between the cells
)

# Add title
plt.title("Heatmap Example")

# Display the plot
plt.show()
```



6.1.4 Pairplots



What is a Pairplot?

A matrix of pairwise scatterplots for all numerical features in a dataset. Diagonal cells show univariate distributions (histograms/KDE). Off-diagonal cells show bivariate relationships between two variables.

Use `seaborn.pairplot()`.

Key Parameters:

`hue='species'`

Color-code points by a categorical column.

`palette='husl'`

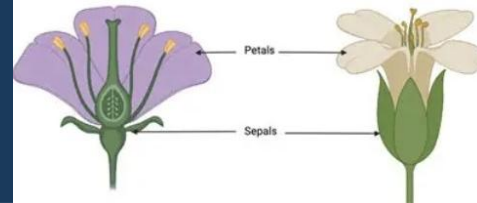
Color palette for different categories.

`sns.load_dataset('iris')`

Load the famous Iris flower dataset.

 *Example (Iris dataset): Compare sepal length, sepal width, and petal length across 3 flower species in one matrix view.*

6.1.4 Pairplots



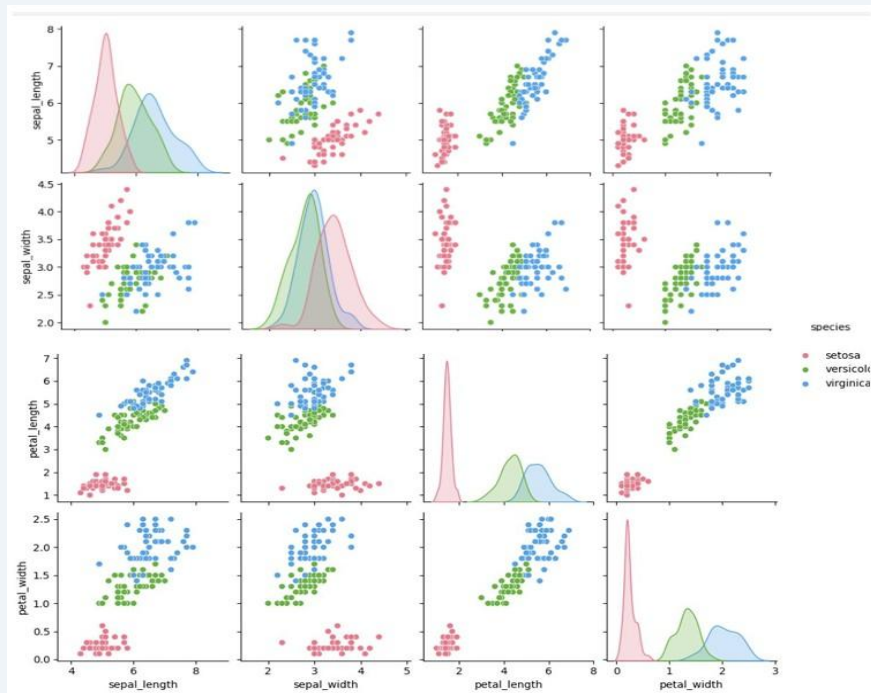
The Code

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load a sample dataset (Iris dataset)
df = sns.load_dataset('iris')

# Create a pairplot
sns.pairplot(
    df,
    hue='species', # Differentiate points by species
    palette='husl' # Color palette
)

# Show the plot
plt.show()
```



Why this code is cool: creates a pair plot, which is great for seeing relationships across an entire dataset at once.

sns.load_dataset('iris'): Seaborn comes with built-in "toy" datasets like this one (data about flower petals and sepals) so you can practice plotting without needing your own CSV file.

sns.pairplot: This automatically creates a grid of scatter plots for every numerical column in your data. It's an incredibly fast way to do **Exploratory Data Analysis (EDA)**.

hue='species': This tells the code to color the dots based on the category (the flower species), making it easy to see clusters in the data.

Choosing the Right Advanced Plot



Histogram

What does the distribution look like?

Use when:

- One continuous variable
- Frequency of values
- Normal vs skewed?

Distribution of customer ages



Boxplot

What is the spread and are there outliers?

Use when:

- Quartile summary
- Comparing groups
- Detecting outliers

Salary ranges across departments



Heatmap

How do values relate in a matrix?

Use when:

- Correlation matrices
- 2D tabular data
- Pattern identification

Feature correlations in ML



Pairplot

How do all variable pairs relate?

Use when:

- Multiple variables
- Group comparison
- Exploratory analysis

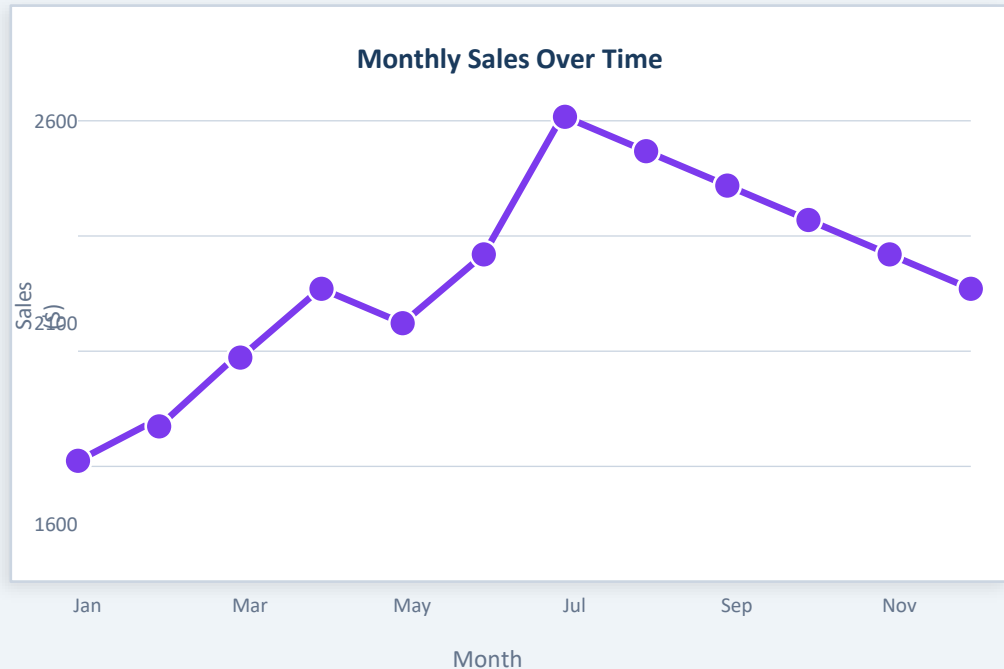
Iris species measurements

Section 6.2

Plotting Time Series Data

Trends · Seasonality · Patterns over Time

6.2 Time Series Visualization



📌 Example: Monthly sales data for 2023 → line plot reveals an upward trend peaking in July, then a gradual seasonal decline toward year-end.

Key Concepts

A time-series plot has time on the x-axis and the measured variable on the y-axis. Used in economics, finance, engineering, and meteorology.

`pd.date_range()`

Create date sequences (monthly, daily, etc.).

`freq='M'`

Frequency of the date range (M = month end).

`marker='o'`

Circle marker at each data point.

`linestyle='-'`

Solid connecting line.

`plt.grid(True)`

Add gridlines for better readability.

`plt.xticks(rotation=45)`

Rotate labels to prevent overlap.

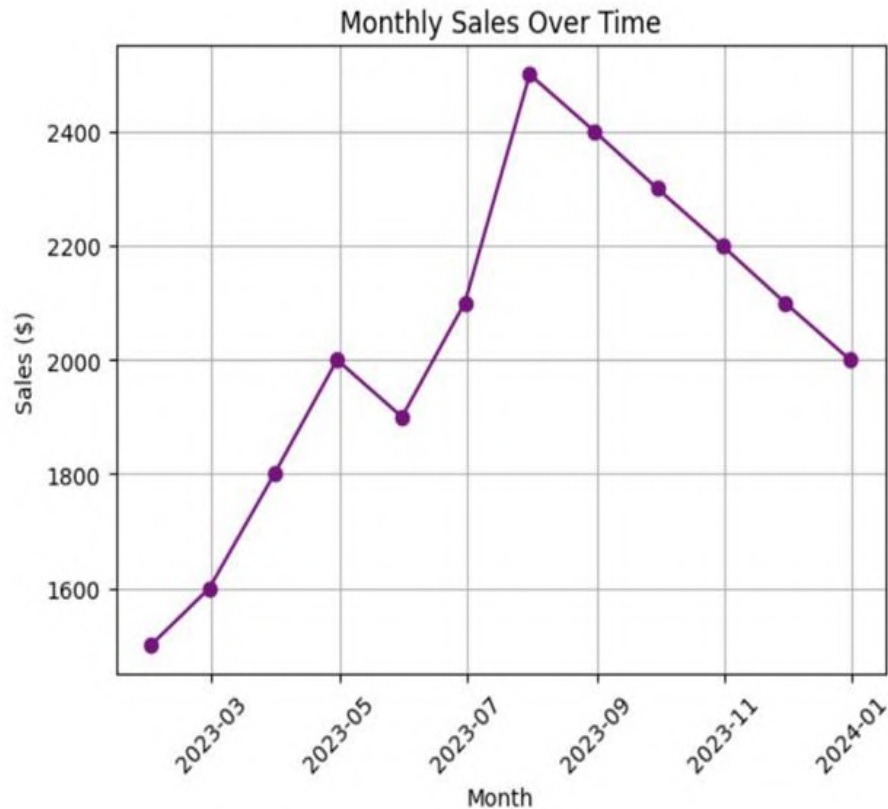
6.2 Time Series Visualization

The Code

```
import matplotlib.pyplot as plt
import pandas as pd

# Sample time series data
dates = pd.date_range(start="2023-01-01", periods=12,
                      freq='M')
sales = [1500, 1600, 1800, 2000, 1900, 2100, 2500, 2400,
        2300, 2200, 2100, 2000]

plt.plot(dates, sales, marker='o', linestyle='-',
        color='purple')
plt.title("Monthly Sales Over Time")
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.grid(True)
plt.xticks(rotation=45)
plt.show()
```



Section 6.3

Best Practices for Effective Visualizations

Clarity · Consistency · Chart Selection · Annotations · Audience Design

6.3 Best Practices — Part 1

6.3.1

Clarity

Purpose:

Ensure the audience easily understands the message.

Best Practices:

- ✓ Use descriptive titles and axis labels
- ✓ Include a legend for multiple categories
- ✓ Avoid overcrowding with too many elements

6.3.2

Consistency

Purpose:

Make visualizations easy to follow and visually uniform.

Best Practices:

- ✓ Use consistent color schemes and styles across plots
- ✓ Add gridlines and markers for readability

6.3.3

Appropriate Visualizations

Purpose:

Match the chart type to the data and context.

Best Practices:

- ✓ Bar charts → comparisons
- ✓ Line plots → trends over time
- ✓ Scatter plots → relationships
- ✓ Histograms → distributions

6.3 Best Practices — Part 2

6.3.4 Annotations

Purpose: Highlight specific data points or trends to emphasize key insights.

Best Practices:

- ✓ Add annotations to emphasize key insights in the data
- ✓ Use arrows (arrowstyle='->') and text boxes for clarity

plt.annotate() Key Arguments:

- xy=(5,50)** Point being annotated.
- xytext=(3,45)** Where text label is placed.
- arrowprops** Arrow color and style.

```
import matplotlib.pyplot as plt
```

```
# 1. Define sample data (needed for the plot to work)
```

```
x = [1, 2, 3, 4, 5, 6, 7]
```

```
y = [10, 25, 35, 42, 50, 45, 30]
```

```
# 2. Create the plot
```

```
plt.plot(x, y, marker='o')
```

```
# 3. Highlighting a specific point
```

```
plt.annotate("Peak Value",
```

```
            xy=(5, 50),    # The point to point to
```

```
            xytext=(3, 45), # Where the text should sit
```

```
            arrowprops=dict(facecolor='red', arrowstyle='->'))
```

```
# 4. Final touches
```

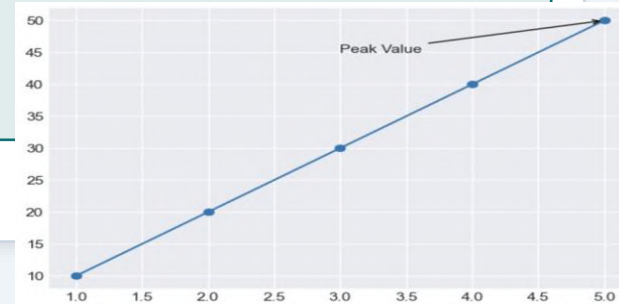
```
plt.title("Data Trend with Annotation")
```

```
plt.xlabel("X-Axis")
```

```
plt.ylabel("Y-Axis")
```

```
# 5. Show the plot
```

```
plt.show()
```



6.3 Best Practices — Part 2

6.3.5 Audience-Centric Design

Purpose: Tailor visualizations to your audience's needs and technical level.

Best Practices:

- ✓ Use simple, clean plots for non-technical audiences
- ✓ Add detailed legends and axis labels for analysts or researchers

Non-Technical Audience →

Simple bar/line charts · Minimal jargon · Bold key numbers · Clear titles

Analysts / Researchers →

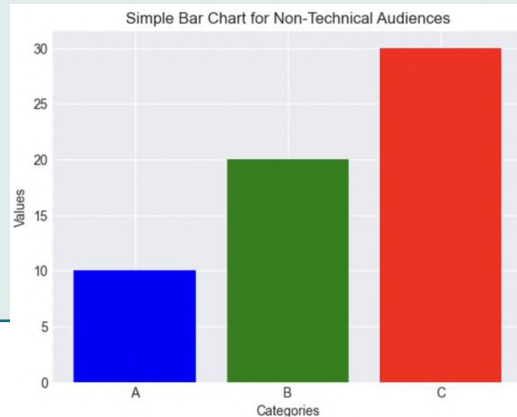
Pairplots, heatmaps · Detailed axes & legends · Statistical annotations

```
import matplotlib.pyplot as plt
```

```
# Create the bar chart with specific colors for each category  
plt.bar(['A', 'B', 'C'], [10, 20, 30], color=['blue', 'green', 'red'])
```

```
# Add descriptive titles and labels  
plt.title("Simple Bar Chart for Non-Technical Audiences")  
plt.xlabel("Categories")  
plt.ylabel("Values")
```

```
# Display the final plot  
plt.show()
```



Chapter 6 — Summary

Topic	Section	Key Takeaway	Library
Histogram	6.1.1	Frequency distribution of continuous data. Key params: bins, color, alpha.	Matplotlib
Boxplot	6.1.2	5-number summary (min, Q1, median, Q3, max). Shows IQR & outliers beyond 1.5×IQR.	Matplotlib
Heatmap	6.1.3	Matrix data with color intensity. Used for correlation matrices. Key: annot, cmap, linewidths.	Seaborn
Pairplot	6.1.4	Matrix of scatterplots for all variable pairs. Diagonal = univariate. hue colors by category.	Seaborn
Time Series	6.2	Time on x-axis. Use pd.date_range(), marker, grid, xticks rotation for clarity.	Matplotlib + Pandas
Clarity	6.3.1	Descriptive titles, axis labels, legend. Avoid overcrowding.	Design
Consistency	6.3.2	Uniform color schemes, gridlines, and markers across plots.	Design
Chart Choice	6.3.3	Bar=compare, Line=trend, Scatter=relationship, Histogram=distribution.	Design
Annotations	6.3.4	plt.annotate() with xy, xytext, arrowprops to highlight key data points.	Matplotlib
Audience Design	6.3.5	Simple plots for non-technical; detailed axes & legends for analysts.	Design

Chapter 6 Complete

Data Visualization (Part 2)

✓ Histograms, Boxplots, Heatmaps, Pairplots

✓ Time Series Visualization

✓ Clarity, Consistency, Annotations & Audience Design

CHAPTER 9

Introduction to Machine Learning in Python

Dr. Yasser ElGeddawy

- 1. Fundamentals of Machine Learning
- 2. Introduction to scikit-learn
- 3. Simple Regression & Classification Models
- 4. Evaluating Model Performance

What You Will Learn in Chapter 9

1

Fundamentals of Machine Learning

Understand core concepts: supervised, unsupervised, and reinforcement learning. Learn about regression, classification, and clustering algorithms.

2

Introduction to scikit-learn

Become familiar with this powerful Python library. Learn to load, preprocess, and split data for training and testing.

3

Simple Regression & Classification Models

Implement linear regression for continuous values. Implement classification models (logistic regression, decision trees) for categorical outcomes.

4

Evaluating Model Performance

Learn metrics: accuracy, precision, recall, F1-score, MSE, and R^2 . Understand cross-validation and preventing overfitting.

9.1.1 What is Machine Learning?

Core Concept

Machine learning focuses on developing systems that can learn from data and make predictions or decisions without being explicitly programmed. These systems identify patterns and relationships within data to make inferences and generalizations.

How ML Enhances Data Analysis:



Predictive Modeling

Forecast future trends and outcomes for proactive decision-making



Data-Driven Insights

Uncover hidden patterns not apparent through traditional analysis



Automation

Automate repetitive tasks: data cleaning, feature engineering, model building



Improved Decision Making

Inform better business decisions in product, marketing, and risk management

Three Major Types of Machine Learning

Supervised Learning

Definition:

Training on labeled data — each point has input features AND a known output (label).

Goal:

Learn a mapping function to predict outputs for new unseen data.

Examples:

- ▶ Regression: Predicting house prices, stock prices
- ▶ Classification: Spam detection, image recognition

Unsupervised Learning

Definition:

Works with unlabeled data — discover hidden patterns, structures, or groupings.

Goal:

Find underlying structure without predefined labels.

Examples:

- ▶ Clustering: Customer segmentation (K-Means)
- ▶ Dimensionality Reduction: PCA, t-SNE

Note: A third type — Reinforcement Learning — trains agents via rewards and penalties (e.g., game-playing AI). Covered in advanced courses.

scikit-learn: Python's Machine Learning Library

scikit-learn is a powerful and widely used Python library for machine learning. It provides a comprehensive set of tools for various machine learning tasks — valuable for both beginners and experts.



Preprocessing Tools

- Handling missing values (mean/median imputation)
- Feature scaling (normalization, standardization)
- Feature selection (reduce dimensionality)
- Encoding categorical variables (e.g., male → 0)



Algorithms

- Regression: Linear, Ridge, Lasso, SVR, Decision Tree, Random Forest
- Classification: Logistic, SVM, KNN, Naive Bayes, Random Forest
- Clustering: K-Means, DBSCAN, Hierarchical
- Dim. Reduction: PCA, t-SNE



Model Evaluation

- Accuracy, Precision, Recall, F1-score
- MSE, R-squared for regression
- k-fold cross-validation
- Train/test split utilities



Hyperparameter Tuning

- Grid Search (GridSearchCV)
- Random Search (RandomizedSearchCV)
- Optimize model parameters
- Prevent overfitting/underfitting

Example: Importing scikit-learn & Checking Version

```
import sklearn  
  
print("scikit-learn version:", sklearn.__version__)
```

Key Points:

scikit-learn is a versatile and user-friendly library for implementing machine learning algorithms.

Its comprehensive set of tools makes it a valuable resource for data scientists and ML practitioners.

By mastering scikit-learn, you can effectively build, train, and evaluate a wide range of machine learning models.

9.3.1 Regression: Predicting Continuous Values

Definition

Regression is a supervised machine learning technique used to predict continuous numerical values.

Common Use Cases:



Predicting house prices based on features like size and location



Estimating sales revenue from advertising spend



Forecasting temperature or stock prices

Supervised Learning Flow:



Steps in Linear Regression:

1

Prepare Data

Define features (X) and target (y)

2

Create Model

`LinearRegression()`

3

Train

`model.fit(X, y)`

4

Predict

`model.predict([new_values])`

Example: Predicting House Prices with Linear Regression

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data: Features (e.g., house size in 1000 sq ft)
X = np.array([[1], [2], [3], [4], [5]]) # Independent variable
                                         # (size in 1000s sq ft)
y = np.array([100, 200, 300, 400, 500]) # Dependent variable
                                         # (price in $1000s)

# Create a Linear Regression model
model = LinearRegression()

# Train the model on the data
model.fit(X, y)

# Make predictions for house sizes 6 and 7 (in 1000 sq ft)
predictions = model.predict([[6], [7]])
print("Predicted Prices (in $1000s):", predictions)
```

Explanation

1. Data

X = house size (1000s sq ft)
y = price (in \$1000s)

2. Model Creation

LinearRegression() fits $X \rightarrow y$ relationship

3. Training

model.fit(X, y) adjusts parameters to best fit data

4. Predictions

model.predict([[6],[7]]) gives output for unseen data

Output Example:

```
Predicted Prices (in $1000s): [600. 700.]
```


9.3.2 Classification: Predicting Discrete Categories

Definition

Classification is a supervised machine learning technique used to predict categorical outcomes (discrete classes).

Regression vs. Classification:

	Regression	Classification
Output Type	Continuous (numbers)	Discrete (categories)
Example Output	House price: \$620,000	Email: Spam / Not Spam
Common Algorithms	Linear Regression	KNN, Logistic Reg, Decision Tree
Evaluation Metric	MSE, R ²	Accuracy, F1-score

Common Use Cases:



Classifying emails as Spam or Not Spam



Diagnosing diseases based on medical test results



Identifying sentiment (positive/negative) in text reviews

Example: Classifying Emails as Spam or Not Spam k-nearest neighbor (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

# Sample data
X = [[1], [2], [3], [4], [5]] # Features (e.g., email length)
y = [0, 0, 1, 1, 1]           # Target (0: Not Spam, 1: Spam)

# Initialize K-Nearest Neighbors Classifier
classifier = KNeighborsClassifier(n_neighbors=3)

# Train the model
classifier.fit(X, y)

# Predict the class of new data points
predictions = classifier.predict([[2.5], [4.5]])
print("Predicted classes:", predictions)
```

Steps

1

Data Preparation

X = email feature (length or word count)
y = 0 (Not Spam) or 1 (Spam)

2

Model Training

KNeighborsClassifier(n_neighbors=3) -> 3 mean select the nearest 3 neighbors
.fit(X, y) trains the model

3

Prediction

.predict() classifies new emails (2.5 and 4.5)

Expected Output:

Predicted classes: [0 1]

Explanation: Email with feature 2.5 → "Not Spam" (0) | Email with feature 4.5 → "Spam" (1)

Example: Classifying Colors with brightness and saturation k-nearest neighbor (KNN)

Example

Consider a dataset with two features: Brightness and Saturation, and two classes: Red and Blue.

Dataset:

Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue

New Data Point:

Brightness	Saturation	Class
20	35	?

How KNN Works

- 1 Choose the value of k:** This determines how many neighbors will be considered for prediction.
- 2 Calculate distances:** Compute the distance between the new data point and all points in the training dataset using a distance metric like Euclidean, Manhattan, or Minkowski.
- 3 Find the k nearest neighbors:** Select the k data points with the smallest distances.
- 4 Make a prediction:** For classification, assign the class that appears most frequently among the k neighbors (majority voting). For regression, take the average of the values of the k neighbors.

Expected Output:

Predict the class: The majority class among the neighbors is Red (2 out of 3), so the new data point is classified as Red.

9.4.1 Metrics for Regression Models

Model evaluation is critical to assess how well a machine learning model performs on unseen data.

1

Mean Absolute Error (MAE)

Measures the average magnitude of errors between true and predicted values. Treats all errors equally.

$$\text{MAE} = \sum |y_{\text{true}} - y_{\text{pred}}| / n$$

2

Mean Squared Error (MSE)

Penalizes larger errors more than smaller ones — squaring amplifies big mistakes. Lower is better.

$$\text{MSE} = \sum (y_{\text{true}} - y_{\text{pred}})^2 / n$$

3

R² Score (Coefficient of Determination)

Range: 0 (poor fit) → 1 (perfect fit). Indicates how much variance in target y is explained by the model.

$$R^2 = \text{Proportion of variance explained by model}$$

Example: Regression Performance Evaluation

```
from sklearn.metrics import mean_squared_error, r2_score

# True vs Predicted values
y_true = [2, 4, 6, 8, 10]
y_pred = [2.1, 3.9, 6.2, 7.8, 9.5]

# Calculate metrics
mse = mean_squared_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

Expected Output:

```
Mean Squared Error: 0.026
R2 Score: 0.9952
```

Explanation

MSE = 0.026

- ▶ Average squared difference between true and predicted.
- ▶ Smaller MSE = better performance.
- ▶ 0.026 means predictions are very close to truth.

R² = 0.9952

- ▶ 99.52% of variance in y_{true} is explained by the model.
- ▶ Value near 1 = excellent fit.
- ▶ Ideal range is 0 $\rightarrow$ 1.

9.4.2 Metrics for Classification Models

1. Accuracy

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

Overall proportion of correct predictions. Good for balanced datasets.

2. Precision

$$\text{Precision} = \text{True Positives} / (\text{TP} + \text{False Positives})$$

How many of the predicted positives were actually positive? Focus: avoid false alarms.

3. Recall (Sensitivity)

$$\text{Recall} = \text{True Positives} / (\text{TP} + \text{False Negatives})$$

How many of the actual positives were correctly identified? Focus: catch all real cases.

4. F1-Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of Precision and Recall. Useful for imbalanced datasets.

Example: Classification Performance Evaluation

```
from sklearn.metrics import accuracy_score, classification_report

# True vs Predicted classes
y_true = [0, 1, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1]

# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
report = classification_report(y_true, y_pred)

print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Expected Output:

```
Accuracy: 0.8
Classification Report:
      precision    recall  f1-score   support

0         0.67      1.00    0.80         2
1         1.00      0.67    0.80         3
```

Accuracy = 0.8 (80%)

4 of 5 predictions are correct.

Classification Report Table:

	Prec.	Recall	F1	Supp.
Class 0	0.67	1.00	0.80	2
Class 1	1.00	0.67	0.80	3
Macro Avg	0.83	0.83	0.80	5
Wt. Avg	0.87	0.80	0.80	5

Macro Avg: Unweighted avg across classes.

Weighted Avg: Weighted by class sample count.

Interpreting the Classification Report

	Precision	Recall	F1-Score	Support
Class 0 (Not Spam)	0.67	1.00	0.80	2
Class 1 (Spam)	1.00	0.67	0.80	3
Accuracy			0.80	5
Macro Avg	0.83	0.83	0.80	5
Weighted Avg	0.87	0.80	0.80	5

Interpretation:



Precision (Class 0 = 0.67):
Of all samples predicted as Class 0, only 67% were actually Class 0.



Precision (Class 1 = 1.00):
All samples predicted as Class 1 were correctly Class 1.



Recall (Class 0 = 1.00):
All actual Class 0 samples were correctly identified as 0.



Recall (Class 1 = 0.67):
Only 67% of actual Class 1 samples were identified correctly.



F1-Score (both = 0.80):
Harmonic mean of Precision and Recall for each class.

Chapter 9 — Summary

9.1 Machine Learning

ML builds systems that learn from data to make predictions without explicit programming. Supervised learning uses labeled data; unsupervised finds hidden patterns. ML enables predictive modeling, automation, and data-driven decisions.

9.2 scikit-learn

A comprehensive Python library offering preprocessing tools (scaling, encoding, imputation), a wide range of algorithms (regression, classification, clustering, PCA), and model evaluation utilities.

9.3 Regression & Classification

Regression predicts continuous values (e.g., house prices using `LinearRegression`). Classification predicts discrete categories (e.g., spam detection using `KNeighborsClassifier`).

9.4 Model Evaluation

Regression: MAE, MSE (lower = better), R^2 ($0 \rightarrow 1$, higher = better). Classification: Accuracy, Precision, Recall, F1-Score. The classification report provides per-class breakdown of all metrics.